

Ejercicio 3

```
class Employee {
private:
    int salary;

public:
    void set_data(int s) {
        salary = s;
    }
    int get_data() {
        return salary;
    }

};

int main() {
    Employee E;
    E.set_data(2500);
    cout << "The salary is: " << E.get_data() << endl;
    return 0;
}
```

Ejercicio 4

```
class Employee {

private:

    int salary;
    string name;

public:

    Employee(string n, float s) {
        name = n;
        salary = s;
    }

    Employee() {
        cout << "Type your name: ";
        cin >> name;
        cout << "Type your salary: ";
        cin >> salary;
    }

}
```

```

void print() {
    cout << "Employee's name: " << name << endl;
    cout << "Employee's salary: " << salary << endl;
}
};

```

```

int main() {
    Employee e;
    Employee w("Willie", 3000);
    e.print();
    w.print();
    return 0;
}

```

Ejercicio 4

```

const double PI = 3.141592;
class Shape {
private: float var; public: Shape(float v) { var = v; }
double circle_area() {
    return PI * var * var;
}

double circle_perimeter() {
    return 2 * PI * var;
}
float square_area() {
    return var * var;
}
float square_perimeter() {
    return 4 * var;
}

};
int main() {
    while (true) {
        cout << "Choose option from the following menu: " << endl;
        cout << "1.Circle\n2.Square\n3.Exit" << endl;
        cout << "Type the option number: ";

        int op;
        cin >> op;

        switch (op) {

```

```

case 1: {
    cout << "Enter radius: ";
    float r;
    cin >> r;
    Shape c(r);
    cout << "The circle area is: " << c.circle_area() << endl;
    cout << "The circle perimeter is: " << c.circle_perimeter() << endl;
    break;
}
case 2: {
    cout << "Enter the side: ";
    float s;
    cin >> s;
    Shape sq(s);
    cout << "The square area is: " << sq.square_area() << endl;
    cout << "The square perimeter is: " << sq.square_perimeter() << endl;
    break;
}
case 3: {
    exit(0);
    //break;
}
default: {
    printf("Invalid option");
}
}
}

```

Ejemplo inheritance 1:

```

#include <iostream>
using namespace std;

class Vehicle {                // Base class
public:
    string brand;
    void show() {
        cout << "Vehicle type: " << brand << endl;
    }
};

class Car : public Vehicle {

```

```

public:
string model;
void set(string m, string b) {
model = m;
brand = b;
}
void print() {
cout << "The car brand is: " << brand << endl;
cout << "The car model is: " << model << endl;
}
};

int main() {
Vehicle v;
Car c;
c.set("Camaro", "Chevy");
c.show();
c.print();
v.brand = "BYD";
cout << "BASE CLASS: " << v.brand << endl;

return 0;
}

```

Ejemplo 2. Inheritance

```

#include <iostream>
using namespace std;

class Mother {
private:
string mother_name;
public:
void set_data(string m) {
mother_name = m;
}

void show() {
cout << "Mother name is: " << mother_name << endl;
}
};

class Daughter : public Mother {
public:

```

```

string daughter_name;
void print() {
    cout << "Daughter name is: " << daughter_name << endl;
}

```

```
};
```

```

int main() {
    Daughter d;
    d.daughter_name = "Maria";
    d.set_data("Carmen");
    d.show();
    d.print();

```

```

    return 0;
}

```

Tercer ejemplo. Inheritance

```

#include <iostream>
using namespace std;

```

```

class Array {
public:
    int nums[5] = { 2, 6, 5, 8, 7 };
    int find(int x){
        for (int i = 0; i < 5; i++) {
            if (x == nums[i]) {}
        }
        return i;
    }
    return -1;
}
};

```

```

class Math : public Array {
public:
    int sum() {
        int s = 0;
        for (int i = 0; i < 5; i++) {
            s += nums[i];
        }
        return s;
    }
}

```

```

int max() {
int m = 0;
for (int i = 0; i < 5; i++) {
if (m < nums[i]) {
m = nums[i];
}

}
return m;
}
};

int main() {
Math p;
cout << "The index is: " << p.find(5) << endl;
cout << "The sumation is: " << p.sum() << endl;
cout << "The maxium number is: " << p.max() << endl;

return 0;
}

```

4rto ejemplo inheritance

```

#include <iostream>
using namespace std;

class Employee {
private:
int id;
protected:
float salary;
public:
string name;

void set(int i) {
id = i;
}

void show() {
cout << "Employee id is: " << id << endl;
}
};

```

```

class Programmer : public Employee {
public:
void set_value(string n, float s) {
    name = n;
    salary = s;
}
void print() {
    cout << "The programmer name: " << name << endl;
    cout << "The porgrammer salary: " << salary << endl;
}

};

```

```

int main() {
    Programmer p;
    p.set(30);
    p.show();
    p.set_value("Carlos", 5300.15);
    p.print();
}

```

Ejercicio de práctica en casa(Inheritance):

```

#include
using namespace std;

class Shape2 {
protected:
    float base;
    float height;
public:
    void set(float b, float h) {
        base = b;
        height = h; }
    void show() {
        cout << "The side length is: " << base << endl;
        cout << "The height length is: " << height << endl;
    }

};

class Triangle : public Shape2 {
public:
    float t_area() {
        return base * height * 0.5;
    }
}

```

```

}

};
class Rectangle : public Shape2 {
public:
float r_area() {
    return base * height;
}

};

int main() {
Triangle t;
Rectangle r;
t.set(3, 5);
r.set(8, 5);
cout << "The triangle area is: " << t.t_area() << endl;
cout << "The rectangle area is: " << r.r_area() << endl;
return 0;
}

```

Ejercicio inheritance multilevel:

```

#include <iostream>
using namespace std;

class Base {
public:
void print() {
    cout << "Base class" << endl;
}
};

class Derived1 : public Base {
public:
void show() {
    cout << "Derived1 class" << endl;
}
};

class Derived2 : public Derived1 {

```

```

public:
void display() {
cout << "Derived2 class" << endl;
}

};

int main() {
Derived2 d2;
d2.print();
d2.show();
d2.display();

return 0;
}

```

Ejercicio multiple inheritance:

```

#include <iostream>
using namespace std;

class Mammal {
public:
void print() {
cout << "Mammals can give direct birth" << endl;
}
};

class WingedAnimal {
public:
void show() {
cout << "Winged animal can fly" << endl;
}
};

class Bat : public Mammal, public WingedAnimal {

};

int main() {
Bat b;
b.print();
}

```

```
b.show();
```

```
return 0;
```

```
}
```

Ejercicio inheritance de constructor sin parámetros y ambos con constructor:

```
#include <iostream>
```

```
using namespace std;
```

```
class Base {
```

```
public:
```

```
Base() {
```

```
cout << "Base class" << endl;
```

```
}
```

```
};
```

```
class Derived : public Base {
```

```
public:
```

```
Derived() {
```

```
cout << "Derived class" << endl;
```

```
}
```

```
};
```

```
int main() {
```

```
Derived d;
```

```
return 0;
```

```
}
```

Ejemplo inheritance cuando base tiene constructor con parámetros y derived no tiene constructor:

```
#include <iostream>
```

```
using namespace std;
```

```
class Base {
```

```
protected:
```

```
int x;
```

```
public:
```

```
Base(int a) {
```

```

x = a;
}

void print() {
cout << "Base class" << endl;
cout << "X: " << x << endl;
}

};

class Derived : public Base {
public:
using Base::Base;
void show() {
cout << "Derived class" << endl;
}
};

int main() {
Derived d(10);
d.print();
d.show();

return 0;
}

```

- Hasta este punto puros ejercicios de inheritance con public, hay con private y protected.

Ejercicio de inheritance con ambos constructores en base y derived y ambos con parámetros

```

#include <iostream>
using namespace std;

class Base {
protected:
float salary;
int id;
public:
Base(float s, int i) {
salary = s;

```

```

id = i;
}
};

class Derived : public Base {
private:
string name;
public:
Derived(string n, float s, int i) : Base(s, i) {
name = n;
}

void print() {
cout << "EMployee's name: " << name << endl;
cout << "EMployee's salary: " << salary << endl;
cout << "EMployee's id: " << id << endl;
}
};

int main() {

Derived d("Carlos", 20000, 101);
d.print();

return 0;
}

```

Ejercicio inheritance cuando la herencia es protected

```

#include <iostream>
using namespace std;

class Base {
private:
string password;
protected:
string user_name;
public:
Base(string p, string u) {
password = p;
user_name = u;
}
}

```

```

};

class Derived : protected Base {
private:
string Email;
public:
Derived(string e, string p, string u) : Base(p, u) {
Email = e;
}
void print() {
cout << "User name is: " << user_name << endl;
cout << "Email: " << Email << endl;
}
};

class Derived2 : public Derived {

};

int main() {
Derived d("Mark@yahoo.com", "1234mark", "Mark95");
d.print();

return 0;
}

```

Ejemplo de override methods (bases del polimorfismo)

```

#include <iostream>
using namespace std;

class Base {
public:
void print() {
cout << "Base class" << endl;
}
};

class Derived : public Base {
public:
void print() {
Base::print(); // Bases del polimorfismo
}
}

```

```
cout << "Derived Class" << endl;
}
};
```

```
int main() {
Derived d;
d.print();
return 0;
}
```

Otro ejemplo:

```
#include <iostream>
using namespace std;
```

```
class Base {
protected:
int x;
public:
void set_value(int a) {
x = a;
}
void print() {
cout << "Base class has " << x << " value." << endl;
}
};
```

```
class Derived : public Base {
private:
int y;
public:
void set_value(int a, int b) {
Base::set_value(a);
y = b;
}
void print() {
Base::print();
cout << "Derived class has " << y << " value" << endl;
}
};
```

```
int main() {
Derived d;
```

```
d.set_value(-5, 99);
d.print();

return 0;
}
```

Ejemplo cuando objects son parameters y pueden ser retornados

```
#include <iostream>
using namespace std;

class Student {
private:
    int id;
    string name;
    float grade;

public:
    Student(int i, string n, float g) {
        id = i;
        name = n;
        grade = g;
    }

    float average(Student s1, Student s2) {
        return(s1.grade + s2.grade) / 2;
    };

};

int main() {
    Student stud1(101, "María", 100);
    Student stud2(102, "José", 90);
    cout << "The average is: " << stud2.average(stud1, stud2);

    return 0;
}
```

Mismo ejercicio pero con un tercer student

```
#include <iostream>
```

```

using namespace std;

class Student {
private:
int id;
string name;
float grade;
public:

Student(){}
Student(int i, string n, int g) {
id = i;
name = n;
grade = g;
}
Student average(Student s1, Student s2) {
Student s3(103, "Carlos", 0);
s3.grade = (s1.grade + s2.grade) / 2;
return s3;
}

void print() {
cout << "Student's name is: " << name << endl;
cout << "Student's id is: " << id << endl;
cout << "Student's grade is: " << grade << endl;
}
};

int main() {
Student stud1(101, "María", 100);
Student stud2(102, "María", 90);
Student stud3;
stud3 = stud3.average(stud1, stud2);
stud3.print();

return 0;

}

```

Ejemplo read and show files

```
#include <iostream>
```

```

#include <fstream>
#include <string>
using namespace std;

int main() {
ifstream f("C:/Users/Emilio T.Castillo/Downloads/file.csv");

string line;

while (getline(f, line)) {

cout << line << endl;

}

f.close();

return 0;

}

```

Ejemplo create files

```

#include <fstream>
#include <iostream>
using namespace std;

int main() {
ofstream f("C:/Users/Emilio T. Castillo/Downloads/newfile.csv");

cout << "How many lines do you want to insert: ";
int lines;
cin >> lines;
int id;
string name;
int grade;

for (int i = 0; i < lines; i++) {
cout << "Enter the id for line " << i << " : ";
cin >> id;
cout << "Enter the name for line " << i << " : ";
cin >> name;
}
}

```

```

cout << "Enter the grade for line " << i << " : ";
cin >> grade;
f << id << ',' << name << ',' << grade << endl;
}
f.close();

return 0;
}

```

Ejemplo create a file with array and then read it with getline

```

#include <iostream>
#include <string>
#include <sstream>
#include <fstream>
using namespace std;

int main() {
    string line = "Carlos, Jose, Maria";
    string value[3];

    int i = 0;
    stringstream ss(line);
    while (getline(ss, value[i], ',')) {
        i++;
    }

    cout << value[1] << endl;

    return 0;
}

```

Ejemplo de usar file y dividirlo en líneas

```

#include <iostream>
#include <string>
#include <sstream>
#include <fstream>
#include <iomanip>
using namespace std;

class Student {

```

```

private:
int id;
string name;
int grade1;
int grade2;

public:
Student(){}
Student(int i, string n, int g1, int g2) {
id = i;
name = n;
grade1 = g1;
grade2 = g2;
}

void design() {
cout << "ID | Name | First Mark | Second Mark" << endl << "-----
-----" << endl;
}

void print() {
cout << setw(5) << left << id << setw(12) << left << name << setw(5) << left << grade1 <<
"\t\t" << setw(5) << left << grade2 << endl;
}
};

int main() {
Student stud[9];
string line;
string value[4];
int j = 0;
int i;

ifstream f("C:/Users/Emilio T. Castillo/Downloads/file.csv");
while (getline(f, line)) {
stringstream ss(line);
i = 0;
while (getline(ss, value[i], ',')) {
i++;
}
stud[j] = Student(stoi(value[0]), value[1], stoi(value[2]), stoi(value[3]));
j++;
}

```

```

}

stud[0].design();

for (int k = 0; k < 9; k++) {
    stud[k].print();
}

return 0;
}

```

Ejemplo memory adress

```

#include <iostream>
#include <string>
#include <sstream>
#include <fstream>
#include <iomanip>
using namespace std;

int main() {

    int x;
    x = 5;
    cout << "X's value: " << x << endl;
    cout << "The memory address of x is: " << &x << endl;

    return 0;
}

```

Ejemplo pointer y memory address

```

#include <iostream>
#include <string>
#include <sstream>
#include <fstream>
#include <iomanip>
using namespace std;

int main() {

    int x;

```

```

x = 5;

int* ptr = &x;

cout << "X's value using the pointer: " << *ptr << endl;
cout << "X's value: " << x << endl;
cout << "X's memmory address: " << &x << endl;
cout << "X's memmory address using pointer: " << ptr << endl;

return 0;
}

```

Ejemplo pointers and arrays

```

#include <iostream>
#include <string>
#include <sstream>
#include <fstream>
#include <iomanip>
using namespace std;

int main() {

int nums[5] = { 1,2,3,4,5 };
int* ptr = nums;
cout << "Nums: " << nums << endl;
cout << "ptr: " << ptr << endl;

for (int i = 0; i < 5; i++) {
cout << ptr[i] << endl;
}

return 0;
}

```

Pasar arrays y pointer a una función o method

- Void Fun (int nums[], int size)
- Int Fun(int *ptr, int size)

```

#include <iostream>
using namespace std;

```

```

class Array {
private:
int size;
int* ptr;
public:
Array(int s) {
size = s;
}

int* read() {
ptr = new int(size);
for (int i = 0; i < size; i++) {
cout << "Enter the value at index: " << i << endl;
cin >> ptr[i];
}
return ptr;
}

int suma(int *ptr) {
int s = 0;
for (int k = 0; k < size; k++) {
s = s + ptr[k];
}
return s;
}

};

int main() {
int sz;
cout << "How many elements do you want to insert in the array: ";
cin >> sz;
Array a(sz);
int* pointer = a.read();
for (int i = 0; i < sz; i++) {
cout << pointer[i] << " ";
}

cout << endl << "The sumation is: " << a.suma(pointer);
return 0;
}

```

Ejercicios de polimorfismo

Primer ejemplo de polimorfismo

```
#include <iostream>
using namespace std;

class MyClass {
public:
void fun(int x) {
cout << "One integer value!!" << endl;
cout << "X's value is: " << x << endl;
}
void fun(double x) {
cout << "One double value!!" << endl;
cout << "X's value is: " << x << endl;
}

void fun(int x, int y) {
cout << "Two integer values!!" << endl;
cout << "X's value is: " << x << endl;
cout << "Y's value is: " << y << endl;
}
};

int main() {
MyClass object;
object.fun(5.24);
cout << " " << endl;
object.fun(5);
cout << " " << endl;
object.fun(4, 7);
cout << " " << endl;
return 0;
}
```

Segundo ejemplo de polimorfismo

```
#include <iostream>
using namespace std;
```

```

class Base {
protected:
int x;
public:
Base(int a) {
x = a;
}

void print() {
cout << "Base class has " << x << " value" << endl;
}
};

class Derived: public Base {
private:
int y;
public:
Derived(int a, int b) :Base(a) {
y = b;
}
void print() {
cout << "Derived class has " << y << " value" << endl;
}

};

int main() {
Derived d(10, 20);
Base *ptr = &d;
ptr->print();
return 0;
}

```

Mas ejemplo polimorphism

```

#include <iostream>
using namespace std;

class Game {
protected:
int attack_power;
public:

```

```

void attack() {
    attack_power = rand() % 100 + 1;
}
};

```

```

class Ninja : public Game {
public:
    void attack() {
        cout << "The ninja attack the enemy with " << attack_power << " attack power" << endl;
    }
};

```

```

class Monster : public Game {
public:
    void attack() {
        cout << "The monster attack the enemy with " << attack_power << " attack power" << endl;
    }
};

```

```

int main() {
    Ninja n;
    Monster m;
    Game* ptr1 = &n;
    Game* ptr2 = &m;
    ptr1->attack();
    n.attack();
    ptr2->attack();
    m.attack();
    return 0;
}

```

Más ejemplos

```

#include <iostream>
using namespace std;

```

```

class Shape {
protected:
    int height;
    int base;

```

```

public:
void area() {
cout << "Enter the height: ";
cin >> height;
cout << "Enter the base: ";
cin >> base;
}

};

class Triangle : public Shape {
public:
void area() {
cout << "The triangle area is: " << height * base / 2 << endl;
}
};

class Rectangle : public Shape {
public:
void area() {
cout << "The rectangle area is: " << height * base << endl;
}
};

int main() {
Triangle t;
Rectangle r;
Shape* ptr1 = &t;
ptr1->area();
t.area();
Shape* ptr2 = &r;
ptr2->area();
r.area();

return 0;
}

```

Ejemplo de virtual methods

```

#include <iostream>
using namespace std;

```

```

class Base {
protected:
int x;
public:
Base(int a) {
x = a;
}
virtual void print() {
cout << "Base class has " << x << " value" << endl;
}
};

class Derived : public Base {
private:
int y;
public:
Derived(int a, int b) : Base(a) {
y = b;
}
void print() {
cout << "Derived class has " << y << " value" << endl;
}
};

int main() {
Derived d(5, 10);
Base* ptr1 = &d;
ptr1->print();
d.print();

return 0;
}

```

Ejemplo polimorphism con abstract class

```

#include <iostream>
using namespace std;

class Parents {
protected:
string mother_name;

```

```

string father_name;
public:
void set_data() {
cout << "Enter the mother's name: " << endl;
cin >> mother_name;
cout << "Enter the father's name: " << endl;
cin >> father_name;
}
virtual void print() = 0;
};

class Kids : public Parents {
private:
string kid_name;
public:
void set_data() {
cout << "Enter the kid's name: " << endl;
cin >> kid_name;
}
void print() {
cout << "The father's name is: " << father_name << endl;
cout << "The mother's name is: " << mother_name << endl;
cout << "The kid's name is: " << kid_name << endl;
}
};

int main() {
Kids k;
Parents* ptr = &k;
ptr->set_data();
k.set_data();
k.print();
return 0;
}

```

Para arreglar los espacios con los strings(para proyecto):

```

#include <iostream>
#include <string>
using namespace std;

int main() {

```

```
string name;
int age;
cout << "Enter the age: ";
cin >> age;
cin.ignore();
cout << "Type your name: ";
getline(cin, name);

if (name == "Emilio Torres") {
    cout << "name: " << name << endl;
    cout << "age: " << age << endl;
}

return 0;
}
```

New C++ class

Intro to recursive function (difference between for loop):

```
#include <iostream>
using namespace std;

void print(int x) {
    if (x > 5) {
        cout << "Done" << endl;
        return;
    }
    else {
        cout << x << endl;
        print(x + 1);
    }
}

void show(int x) {
    for (int i = x; i <= 5; i++) {
        cout << i << endl;
    }
    cout << "Done " << endl;
}
```

```

int fact(int num) {
if (num == 1) {
return 1;
}
else {
return num * fact(num - 1);
}
}

```

```

int main() {
print(1);
cout << fact(6) << endl;
return 0;
}

```

Ejemplo más comple de recursive functions:

```

#include <iostream>
#include <string>
using namespace std;

```

```

int count(string s, char x, int index = 0) {
if (index > s.length()) {
return 0;
}
char c = tolower(s[index]);
if (c == x) {
return 1 + count(s, x, index + 1);
}
else {
return 0 + count(s, x, index + 1);
}
}

```

```

int main() {
string s = "Hello World";
cout << count(s, 'l') << endl;
return 0;
}

```

Otro ejercicio:

```

#include <iostream>
#include <string>
using namespace std;

```

```

int count(string s, char x, int index = 0) {
    if (index > s.length()) {
        return 0;
    }
    char c = tolower(s[index]);
    if (c == x) {
        return 1 + count(s, x, index + 1);
    }
    else {
        return 0 + count(s, x, index + 1);
    }
}

```

```

int main() {
    string s = "Hello World";
    cout << count(s, 'l') << endl;
    return 0;
}

```

Fibonnacci with recursion

```

#include <iostream>
using namespace std;

```

```

int sum(int end) {
    if (end <= 0)
        return 0;
    return end + sum(end - 1);
}

```

```

int arr(int* ptr, int size) {
    if (size <= 0) {
        return 0;
    }
    else {
        return ptr[size-1] + arr(ptr, size-1);
    }
}

```

```

int fibonacci(int n) {
    if (n <= 1)
        return n;
    return (fibonacci(n - 1) + fibonacci(n - 2));
}

```

```
int main() {
    cout << sum(5) << endl;
    int nums[10] = { 5,10,15,20,25,30,35,40,45,50 };
    int* ptr = nums;
    cout << arr(ptr, 10) << endl;
```

```
for(int i = 0; i <= 5; i++)
    cout << fibonacci(i) << " " << endl;
return 0;
}
```

Ejemplos de complexity(worst case):

```
#include using namespace std;
```

```
int fun(int n) { int s = 0; for (int i = 0; i <= n; i++) { // O(n) for (int j = 0; j <= n; j++) { // O(n) for (int
k = 0; k < j; k++) { //O(n)
```

```
        }
    }
}
```

```
} // => nnn = n^3 => O(n^3)
```

```
void funny(int n) { if (n > 5) {
```

```
    while (n > 5) { // O(n) n -= 1; cout << n << endl; } } else { for (int i = 0; i <= n; i * 2) { // O(log2 n)
    cout << i; } } }
```

```
void system(int n) { for (int i = 0; i <= n / 2; i++) { // O(n) for (int j = 0; j < 1000; j++) { // O(1) =>
n*1 = O(n)
```

```
    }
}
```

```
}
```

```
int main() { return 0; }
```

Ejemplo ver complexity o Big O de un code:

```
#include using namespace std;
```

```

int fun(int n) { int s = 0; for (int i = 0; i <= n; i++) { // O(n) for (int j = 0; j <= n; j++) { // O(n) for (int
k = 0; k < j; k++) { //O(n)
    }
    }
}

```

```

} // => nnn = n^3 => O(n^3)

```

```

void funny(int n) { if (n > 5) {

```

```

    while (n > 5) { // O(n) n -= 1; cout << n << endl; } } else { for (int i = 0; i <= n; i * 2) { // O(log2 n)
    cout << i; } } }

```

```

void system(int n) { for (int i = 0; i <= n / 2; i++) { // O(n) for (int j = 0; j < 1000; j++) { // O(1) =>
n*1 = O(n)

```

```

    }
}

```

```

}

```

```

int main() { return 0; }

```

Ejemplo complexity factorial with recursive function

```

#include <iostream>
using namespace std;

```

```

void fun(int n) {
    int f = 1;
    if (n > 0) {
        for (int i = n; i > 0; i--) {
            f = f * i;
        }
        cout << f << " ";
        fun(n - 1);
    }
}

```

```
int main() {  
    fun(5);  
    return 0;  
}
```

Guardar files -> lines en un vector

```
#include <iostream>  
#include <vector>  
#include <fstream>  
#include <string>  
using namespace std;  
  
int main() {  
    ifstream f("C:/Users/gking/Downloads/tracks.txt");  
    string line;  
    vector <string> vl;  
  
    while (getline(f, line)) {  
        vl.push_back(line);  
    }  
  
    for (string i:vl) {  
        cout << i << endl;  
    }  
    return 0;  
}
```

Capacity and size, how this grows and its O(n)

```
#include <iostream>  
#include <vector>  
  
using namespace std;  
  
int main(){  
    vector <int> v;  
    cout<<"Size "<<v.size()<<endl;  
    cout<<"Capacity: "<<v.capacity()<<endl;  
    v.push_back(5);  
    v.push_back(10);  
    v.push_back(20);
```

```

    cout<<"Size "<<v.size()<<endl;
    cout<<"Capacity: "<<v.capacity()<<endl;

//Big O(n) if the capacity is full then you copy the values and extend the capacity in another
site

    return 0;

}

```

Ejemplo vector con class y files y sstream (dividir líneas en más líneas):

```

#include <iostream>
#include <string>
#include <vector>
#include <iomanip>
#include <fstream>
#include <sstream>

using namespace std;

class Play {
private:
    int id;
    string name;
    int length;
    int playedCount;
public:
    Play(int i, string n, int l, int p){
        this->id = i;
        this->name = n;
        this->length = l;
        this->playedCount = p;
    }
    void design() {

        cout << "ID | Name | Length | Replay Count" << endl;
        cout << "-----" << endl;
    }

    void display() {

```

```

cout << setw(3) << left << id << " " << setw(10) << left << name << setw(3) << left << length
<< setw(2) << left << playedCount << endl;
}

```

```

};

```

```

int main() {
ifstream file("C:/Users/Usuario Final/Downloads/tracks.txt");
string line;
string values[4];
vector<Play> tracks;
int i;

while (getline(file, line)) {
stringstream ss(line);
i = 0;
while (getline(ss, values[i], ' ')) {
i++;
}
tracks.push_back(Play(stoi(values[0]), values[1], stoi(values[2]), stoi(values[3])));
}
tracks[0].design();
for (int i = 0; i < tracks.size(); i++)
tracks[i].display();
return 0;
}

```

Linear search, binary search and recursive binary search:

```

#include <iostream>
#include <vector>
using namespace std;

int LinearSearch(vector<int> &x, int key) {
for (int i = 0; i < x.size(); i++) {
if (x[i] == key)
return i;
}
return -1;
}

```

```

int binary_search(vector<int>& x, int key) {
int low = 0;
int high = x.size() - 1;
int mid;
while (low <= high) {
mid = (low + high) / 2;
if (key == x[mid])
return mid;
else if (key < x[mid])
high = mid - 1;
else if (key > x[mid])
low = mid + 1;
}
return -1;
}

```

```

int binary_search2(vector<int>& x, int key,int low,int high) {
int mid = (low + high) / 2;
if (low <= high) {

if (key == x[mid])
return mid;
else if (key < x[mid])
return binary_search2(x, key, low, mid - 1);
else if (key > x[mid])
return binary_search2(x, key, mid + 1, high);

}
return -1;

}

```

```

int main() {
//Linear Search
vector<int> y = { 1,2,3,4,5,6,7,8 };
cout << "Index is: " << LinearSearch(y, 4) << endl;

vector<int> z = { 5,7,9,10,11,20,25,30,35,37,40 };

//Recursion Binary Search
int high = z.size() - 1;

```

```
cout << "Index is: " << binary_search2(z, 35, 0, high);
return 0;
}
```

Sorting algorithm:

implementation

```
Void selection_sort(vector <int> &nums){
    int min;
    for(int i = 0; i < nums.size()-1 ; i++){
        min= i;
        for(int j = i+1; j<nums.size(); j++){
            if (nums[j]<nums[min])
                min = j;
        }
        swap(nums[i],nums[min]);
    }
}
```

Handwritten diagrams illustrating the selection sort process:

- Initial array: [5, 3, 8, 1] with indices 0, 1, 2, 3. Arrows point from index 0 to 1 and from index 2 to 3.
- Step 1: [1, 3, 8, 5] with indices 0, 1, 2, 3. Arrows point from index 0 to 1 and from index 2 to 3.
- Step 2: [1, 3, 8, 5] with indices 0, 1, 2, 3. Arrows point from index 0 to 1 and from index 2 to 3.
- Step 3: [1, 3, 8, 5] with indices 0, 1, 2, 3. Arrows point from index 0 to 1 and from index 2 to 3.

Both algorithms:

Handwritten diagrams illustrating the selection sort process:

- Initial array: [5, 3, 8, 1] with indices 0, 1, 2, 3. Arrows point from index 0 to 1 and from index 2 to 3.
- Step 1: [1, 3, 8, 5] with indices 0, 1, 2, 3. Arrows point from index 0 to 1 and from index 2 to 3.
- Step 2: [1, 3, 8, 5] with indices 0, 1, 2, 3. Arrows point from index 0 to 1 and from index 2 to 3.
- Step 3: [1, 3, 8, 5] with indices 0, 1, 2, 3. Arrows point from index 0 to 1 and from index 2 to 3.

Handwritten diagrams illustrating the bubble sort process:

- Initial array: [5, 3, 8, 1] with indices 0, 1, 2, 3. Arrows point from index 0 to 1 and from index 2 to 3.
- Step 1: [1, 3, 8, 5] with indices 0, 1, 2, 3. Arrows point from index 0 to 1 and from index 2 to 3.
- Step 2: [1, 3, 8, 5] with indices 0, 1, 2, 3. Arrows point from index 0 to 1 and from index 2 to 3.
- Step 3: [1, 3, 8, 5] with indices 0, 1, 2, 3. Arrows point from index 0 to 1 and from index 2 to 3.

Examples bubble and selective sort:

```

#include <iostream>
#include <vector>
using namespace std;

void selection_sort(vector<int>& nums) {
    int min;
    int n = nums.size();
    for (int i = 0; i < n - 1; i++) {
        min = i;
        for (int j = i + 1; j < n; j++) {
            if (nums[j] < nums[min])
                min = j;
        }
        swap(nums[i], nums[min]);
    }
}

```

```

void bubble_sort(vector<int>& nums) {
    int n = nums.size();
    bool check_swap;
    for (int i = 0; i < n - 1; i++) {
        check_swap = false;
        for (int j = 0; j < n - i - 1; j++) {
            if (nums[j] > nums[j + 1]) {
                swap(nums[j], nums[j + 1]);
                check_swap = true;
            }
        }
        if (!check_swap)
            break;
    }
}

```

```

void print(vector<int>& nums) {
    for (int i = 0; i < nums.size(); i++) {
        cout << nums[i] << " ";
    }
    cout << endl;
}

```

```

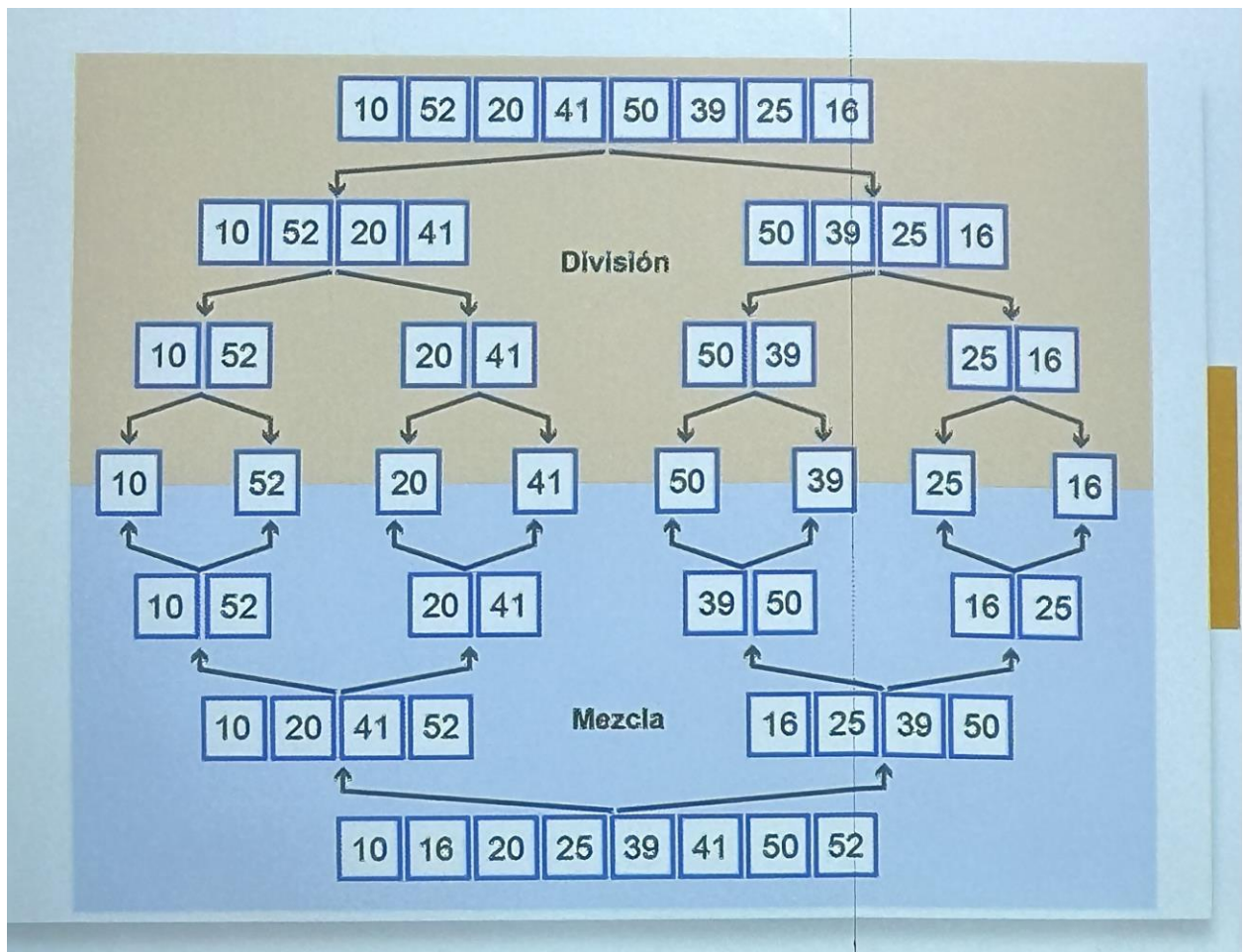
int main() {
vector<int> x = { 5,1,8,3,9,2,7 };
bubble_sort(x);
print(x);

return 0;
}

```

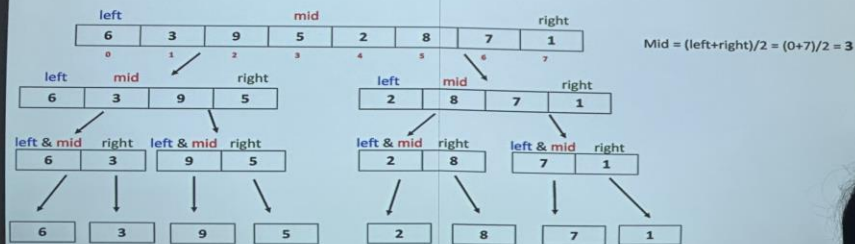
Merge algorithm

- Memory efficient and stable
- Parecido al binary search
- Divides list in halves until it can't be divided further more.

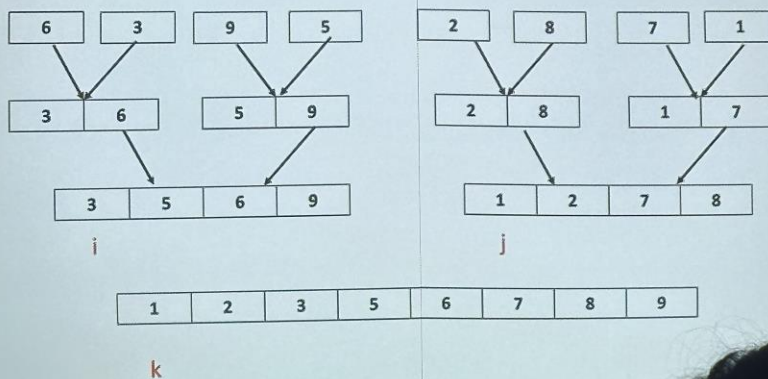


- **CONDITION:** while right doesn't reach left or surpasses it, it will continue to divide the array inputs.

Example 3: tips for implementation, partition task



Merge task:



Codes de linear y binary search:

```
#include <iostream>
#include <vector>
using namespace std;
```

```
int LinearSearch(vector<int>& x, int key) {  
    for (int i = 0; i < x.size(); i++) {  
        if (x[i] == key)  
            return i;  
    }  
    return -1;  
}
```

```
int binary_search(vector<int>& x, int key) {  
    int low = 0;  
    int high = x.size() - 1;  
    int mid;
```

```
    while (low <= high) {  
        mid = (low + high) / 2;  
        if (key == x[mid])  
            return mid;  
        else if (key < x[mid])  
            high = mid - 1;  
        else if (key > x[mid])  
            low = mid + 1;  
    }  
    return -1;  
}
```

```
int binary_search2(vector<int>& x, int key, int low, int high) {  
    int mid = (low + high) / 2;
```

```
    if (low <= high) {  
        if (key == x[mid])  
            return mid;
```

```
        else if (key < x[mid])  
            return binary_search2(x, key, low, mid - 1);
```

```
        else if (key > x[mid])  
            return binary_search2(x, key, mid + 1, high);  
    }  
    return -1;  
}
```

```

int main() {

//Linear Search

vector<int> y = { 1,2,3,4,5,6,7,8 };

cout << "Index is: " << LinearSearch(y, 4) << endl;


vector<int> z = { 5,7,9,10,11,20,25,30,35,37,40 };


//Recursion Binary Search

int high = z.size() - 1;

cout << "Index is: " << binary_search2(z, 35, 0, high);

return 0;

}

```

- Stable algorithm porque preserva el orden relativo de los elementos que son iguales.
- Code diseñada para funcionamiento paralel performance. Esto significa que trabaja en hilos a la vez
- Complexity no es tan bueno por el trabajo con temporary arrays

Merge sort code

```

#include <iostream>
#include <vector>
#include <string>
using namespace std;

void merge(vector<int>& nums, int left, int mid, int right) {
int n1 = mid - left + 1;
int n2 = right - mid;
vector<int> l(n1);
vector<int> r(n2);

```

```

for (int i = 0; i < n1; i++) {
    l[i] = nums[left + i];
}
for (int i = 0; i < n2; i++) {
    r[i] = nums[mid+1 + i];
}
int i = 0;
int j = 0;
int k = left;

```

```

while (i < n1 && j < n2) {
    if (l[i] <= r[j]) {
        nums[k] = l[i];
        i++;
    }
    else {
        nums[k] = r[j];
        j++;
    }
    k++;
}
while (i < n1) {
    nums[k] = l[i];
    i++;
    k++;
}
while (j < n2) {
    nums[k] = r[j];
    k++;
    j++;
}
}

```

```

void merge_sort(vector<int> &nums, int left, int right) {
    if (left >= right)
        return;

```

```

    int mid = (left + right) / 2; //Same as: (left+right)/2
    merge_sort(nums, left, mid);
    merge_sort(nums, mid+1, right);
    merge(nums, left, mid, right);

```

```
}
```

```
void print(vector<int>& nums) {  
    for (int i = 0; i < nums.size(); i++) {  
        cout << nums[i] << " ";  
    }  
    cout << endl;  
}
```

```
int main() {  
    vector<int> nums = { 3,1,6,9,2,5,17,4,8,21 };  
    int n = nums.size();  
    vector<int> x(10); //If you add a second parameter in the parenthesis u can set a value for  
    all the positions
```

```
    cout << "The vector before the sort algorithm: ";  
    print(nums);  
    cout << endl;
```

```
    merge_sort(nums, 0, n - 1);  
    cout << "The vector after the sort algorithm: ";  
    cout << endl;  
    print(nums);
```

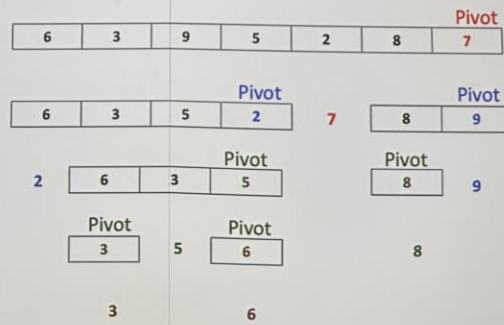
```
    return 0;
```

```
}
```

- Inefficient for large datasets, time complexity = $O(n^2)$

Quick sort:

Example:



Insertion and quick sort:

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;

void insertion_sort(vector<int>& nums) {
    int curr;
    int prev;
    for (int i = 1; i < nums.size(); i++) {
        curr = nums[i];
        prev = i - 1;

        while (prev >= 0 && nums[prev] > curr) {
            nums[prev + 1] = nums[prev];
            prev--;
        }
        nums[prev + 1] = curr;
    }
}
```

```

int partition(vector<int>& nums, int low, int high) {
    int pivot = nums[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (nums[j] < pivot) {
            i++;
            swap(nums[i], nums[j]);
        }
    }
    swap(nums[i + 1], nums[high]);
    return (i + 1);
}

```

```

void quick_sort(vector<int>& nums, int low, int high) {
    if (low < high) {
        int pi = partition(nums, low, high);
        quick_sort(nums, low, pi - 1);
        quick_sort(nums, pi, high);
    }
}

```

```

void print(vector<int>& nums) {
    for (int i = 0; i < nums.size(); i++) {
        cout << nums[i] << " ";
    }
    cout << endl;
}

```

```

int main() {
    vector<int> nums = { 3,1,6,9,2,5,17,4,8,21 };
    int n = nums.size();
    vector<int> x(10); //If you add a second parameter in the parenthesis u can set a value for
    all the positions

```

```

    cout << "The vector before the sort algorithm: ";
    print(nums);
    cout << endl;

```

```

    cout << "The vector after the sort algorithm: ";
    quick_sort(nums,0,n-1);
    print(nums);
    cout << endl;

```

```
return 0;
```

```
}
```

Ejemplos de repaso pointer:

```
#include <iostream>
#include <vector>
using namespace std;
```

```
int* read(int size) {
    int* ptr = new int[size];
    for (int i = 0; i < size; i++) {
        cout << "Enter the element: ";
        cin >> ptr[i];
    }
    return ptr;
}
```

```
void print(int* ptr,int size) {
    for (int i = 0; i < size; i++)
        cout << ptr[i] << " ";
```

```
    cout << endl;
}
```

```
int main() {
```

```
    int x = 5;
    int* ptr = &x;
    cout << "Memory address of x: " << ptr << endl;
    cout << "Value of x accessed from pointer: " << *ptr << endl;
    *ptr = 10;
    cout << "The x value after update is: " << x << endl;
```

```
    int nums[3] = { 1,4,7 };
    cout << nums << endl;
    print(nums, 3);
```

```
return 0;
}
```

Ejemplo how to fix dangeling pointer: cuando nosotros tenemos un memory address que está dentro de una función, solución es meterlo al hip

```
#include using namespace std;
```

```
int* fun() { int* x = new int(5 * 5); // Para usar hip se necesita de ahuevo el pointer return x; }
```

```
int main() { int* ptr = fun(); cout << ptr << endl; delete ptr; return 0; }
```

Ejemplos void pointer, pointer to pointer and function pointer:

```
#include using namespace std;
```

```
float pi = 3.1416;
```

```
//For function pointer int mult(int x, int y) { return x * y; }
```

```
int main() { //Void pointer void* ptr; float x = 5.1; int s = 345; ptr = &x; cout << (float)ptr << endl; ptr = &s; cout << (int)ptr << endl;
```

```
//Pointer to pointer
```

```
int y = 10;
```

```
int* ptr1 = &y;
```

```
int** p = &ptr1;
```

```
cout << **p << endl;
```

```
cout << p << endl;
```

```
//Function pointer
```

```
int(*ptr2)(int, int);
```

```
ptr2 = mult;
```

```
cout << ptr2(5, 7) << endl;
```

```
return 0;
```

```
}
```

Constant pointer

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
int x = 5;
```

```
int y = -10;
```

```
int* const ptr = &x;
```

```
cout << *ptr << endl;
```

```
ptr = &y;
```

```
cout << *ptr << endl;
```

```
}
```

Dynamic memory allocated (se ocupa pointer):

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {  
  
    int* ptr = new int;  
  
    *ptr = 50;  
  
    cout << "Value ptr points to: " << *ptr << endl;  
  
    cout << "Ptr points to his memory address: " << ptr << endl;  
  
  
    delete ptr;                //Borra espacio a donde apunta ptr  
  
    //cout << *ptr << endl;  
  
    //(esto marca error ya que borraste el valor apuntado)  
  
    ptr = nullptr;  
  
    cout << "Ptr points to his memory address: " << ptr << endl;  
  
    cout << "\n" << endl;  
  
    int* ptr1 = new int[5];  
  
    for (int i = 0; i < 5; i++)  
  
        ptr1[i] = i * 10;  
  
  
    for (int i = 0; i < 5; i++) {  
  
        cout << ptr1[i] << " ";  
  
    }  
  
    cout << endl;  
  
  
    delete[] ptr1;
```

```
ptr1 = nullptr;
```

```
return 0;
```

```
}
```

Stack:

```
#include <iostream>
```

```
#include <stack>
```

```
using namespace std;
```

```
int main() {
```

```
    stack<int> s;
```

```
    s.push(5);
```

```
    s.push(10);
```

```
    s.push(15);
```

```
    s.push(20);
```

```
    s.push(25);
```

```
    cout << "Top: " << s.top() << endl;
```

```
    s.pop();
```

```
    cout << "Top: " << s.top() << endl;
```

```
    if (s.empty())
```

```
cout << "The stack is empty" << endl;

else {

cout << "Stack not empty" << endl;

cout << "The size of stack is:" << s.size() << endl;

int x = s.size();

for (int i = 0; i <= x; i++) {

cout << "Top: " << s.top() << endl;

s.pop();

}

}

return 0;

}
```

POO implementation of stack:

```
#include <iostream>
#define MAX 100

using namespace std;

class Stack {
private:
    int array[MAX];
    int top;

public:
    Stack() {
        top = -1;
    }

    bool empty() {
        return (top == -1);
    }

    bool full() {
        return (top == MAX - 1);
    }

    void push(int x) {
        if (full())
            cout << "The stack is full" << endl;
        else {
            top++;
            array[top] = x;
        }
    }

    void pop() {
        if (empty())
            cout << "The stack is empty" << endl;
        else
            top--;
    }

    int Top() {
```

```

    if (empty()) {
        cout << "The stack is empty" << endl;
        return -1;
    }
    else {
        return array[top];
    }
}

int size() {
    if (empty()) {
        cout << "The stack is empty " << endl;
        return 0;
    }
    return top + 1; // esto es igual a top ++ return top;
}
};

int main() {
    Stack s;
    for (int i = 5; i < 35; i = i + 5) {
        s.push(i);
    }
}

```

Usando library de queue:

```

#include <iostream>
#include <queue>
using namespace std;

void printQ(queue <int>& q) {
    while (!q.empty()) {
        cout << "Front-->" << q.front() << endl;
        q.pop();
    }
}

int main() {
    queue <int> myQ;

```

```

for (int i = 5; i < 30; i = i + 5) {
    myQ.push(i);
}

cout << "Front-->" << myQ.front() << endl;
myQ.pop();
cout << "Front-->" << myQ.front() << endl;
cout << "Front-->" << myQ.front() << endl;
cout << "Rear-->" << myQ.back() << endl;

cout << "\n" << endl;
cout << "Using function" << endl;
cout << "Queue's size: " << myQ.size() << endl;
printQ(myQ);
cout << "Again" << endl;
cout << "Queue's size: " << myQ.size() << endl;
printQ(myQ);
return 0;
}

```

Our queue implementation with POO:

```

#include <iostream>
#define Max 100
using namespace std;
template <typename T>

class Queue {
private:
    T arr[Max];
    int front;
    int rear;
public:
    Queue() {
        front = -1;
        rear = -1;
    }
    bool isEmpty() {
        return (front == -1 || front > rear);
    }
    bool isFull() {
        return (rear == Max - 1);
    }

```

```

}
void enqueue(T x) {
    if (isFull())
        cout << "The queue is full!" << endl;
    else {
        if (front == -1) {
            front = 0;
        }
        rear++;
        arr[rear] = x;
    }
}

void dequeue() {
    if (isEmpty())
        cout << "The queue is already empty!" << endl;
    else
        front++;
}

T Front() {
    if (isEmpty()) {
        cout << "The queue is empty" << endl;
        return T();
    }
    else
        return arr[front];
}

T Rear() {
    if (isEmpty()) {
        cout << "The queue is empty" << endl;
        return T();
    }
    else
        return arr[rear];
}

int size() {
    if (isEmpty()) {
        cout << "The queue is empty!!" << endl;
        return 0;
    }
    else
        return (rear - front) + 1;
}

```

```

void printQ() {
while (!isEmpty()) {
cout << Front() << " ";
dequeue();
}
cout << endl;
}
};

```

```

int main() {
Queue <char> q;
q.enqueue('A');
q.enqueue('B');
q.enqueue('C');
q.enqueue('D');
q.enqueue('E');

```

```

q.printQ();

```

```

/*cout << "Front-->" << q.Front() << endl;
cout << "Rear-->" << q.Rear() << endl;
q.dequeue();
cout << "Front-->" << q.Front() << endl;
cout << "Queue size: " << q.size() << endl;*/
return 0;
}

```

Linked list, los basics para usarlo:

```

#include <iostream>
#include <list>
#include <algorithm>
using namespace std;

```

```

void printList(list <int>& l) {
cout << "Head --> ";
for (list <int> :: iterator ptr = l.begin(); ptr != l.end(); ptr++)
cout << *ptr << "-->";

```

```

cout << " NULL" << endl;
}

```

```

int main() {

list<int> ml;
for (int i = 10; i < 60; i = i + 10) {
ml.push_back(i);
}
printList(ml);
list<int> ::iterator ptr = ml.begin();
advance(ptr, 3);
ml.insert(ptr, 35); // Este es el insert
printList(ml);
cout << "Front -->" << ml.front() << endl;
cout << "Back -->" << ml.back() << endl;
cout << "Next -->" << *next(ml.begin(),3) << endl;
//Acá el modify
list<int> ::iterator ptr1 = ml.begin();
advance(ptr1, 4);
*ptr1 = 45;
printList(ml);

//Find element
cout << "\n" << endl;
/*list<int> ::iterator ptr2 = find(ml.begin(),ml.end(),10);
if (ptr != ml.end())
cout << *ptr << endl;
else
cout << "Value not found" << endl;*/

//Pops the front y back
cout << "\n" << endl;
ml.pop_front();
ml.pop_back();
printList(ml);

//Erase element from list
cout << "\n" << endl;
list<int> ::iterator ptr3 = ml.begin();
printList(ml);
advance(ptr, 1);
//erase()

```

```
return 0;
}
```

Ejemplo con linked list:

```
#include<iostream>
#include<list>
using namespace std;

void GradeClassification(list<int>& g, list<int>& f, list<int>& p) {
for (list<int> ::iterator ptr = g.begin(); ptr != g.end(); ptr++) {
if (*ptr >= 70)
p.push_back(*ptr);
else
f.push_back(*ptr);
}
}

void printList(list<int>& l) {
cout << "Head --> ";
for (list<int> ::iterator ptr = l.begin(); ptr != l.end(); ptr++) {
cout << *ptr << " --> ";
}
cout << "NULL" << endl;
}

int main() {
list<int> grades = { 100,90,55,70,60,22,45,85,97,58,10 };
list<int> fail;
list<int> pass;

printList(grades);
GradeClassification(grades,fail,pass);
printList(fail);
printList(pass);
}
```

Linked list class implementation:

```
#include<iostream>
using namespace std;
```

```

class Node {
public:
int value;
Node* next;

Node() {
value = 0;
next = NULL;
}
Node(int v) {
value = v;
next = NULL;
}

};

class Linkedlist {
private:
Node* head;
public:
Linkedlist() {
head = NULL;
}
void push_front(int value) {
Node* new_node = new Node(value);
if (head == NULL) {
head = new_node;
return;
}
else {
new_node->next = head;
head = new_node;
return;
}
}

void push_back(int value) {

if (head == NULL) {
push_front(value);
}
else {

```

```
Node* new_node = new Node(value);
Node* ptr = head;
```

```
while (ptr->next != NULL) {
    ptr = ptr->next;
}
```

```
ptr->next = new_node;
}
}
```

```
void printlist() {
    Node* ptr = head;
    cout << "HEAD --> ";
    while (ptr != NULL) {
        cout << ptr->value << " -->";
        ptr = ptr->next;
    }
    cout << "NULL" << endl;
}
```

```
int size() {
    Node* ptr = head;
    int s = 0;
    while (ptr != NULL) {
        s++;
        ptr = ptr->next;
    }
    return s;
}
```

```
void insert(int index, int value) {
    if (head == NULL) {
        cout << "Linked List is empty" << endl;
    }
    else if (index > size()) {
        cout << "Index of of range!!" << endl;
    }
    else if (index == size()) {
        push_back(value);
    }
    else {
```

```

Node* new_node = new Node(value);
Node* ptr = head;
for (int i = 1; i <= index; i++) {
    ptr = ptr->next;
}
new_node->next = ptr->next;
ptr->next = new_node;

}
}

};
int main() {
    LinkedList l;
    l.push_back(10);
    l.push_back(15);
    l.push_back(20);
    l.push_back(30);
    l.push_back(40);
    l.push_front(0);

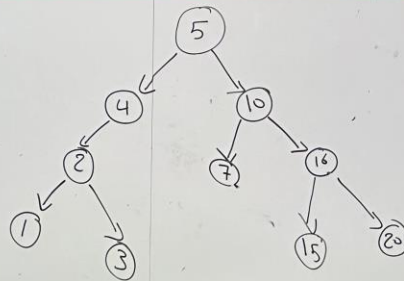
    l.printlist();
    cout << "The size of your Linked List is: " << l.size() << endl;
    l.insert(3, 100);
    l.printlist();
    cout << "The size of your Linked List is: " << l.size() << endl;
    return 0;
}

```

Double linked list: Checar activity2.2

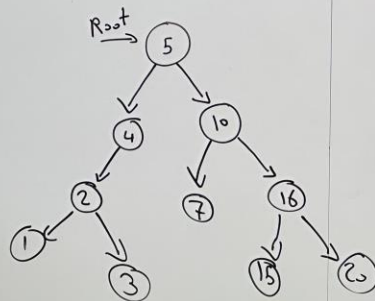
Link trees y ejemplos para armarlos y ordenarlos.

Example1: 5, 10, 4, 2, 16, 7, 1, 20, 15, 3



Ejemplo de crear linktrees a partir de una lista:

Example1: 5, 10, 4, 2, 16, 7, 1, 20, 15, 3



Para entender bien los tipos de orden en trees en código:

```
main.cpp
46 parent->right = new_node;
47 }
48
49
50 void inorder(Node *ptr){
51     if(ptr == nullptr)
52         return;
53     inorder(ptr->left);
54     cout<<ptr->key<<" ";
55     inorder(ptr->right);
56 }
```

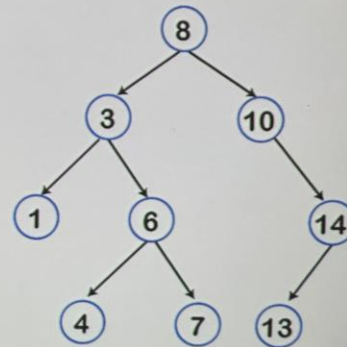
Inorder traversal: 8 10 12 15 20 25

Program finished with exit code 0

Practicar el delete Node en trees:

Node deletion

- BST has three cases :
 1. If node is leaf node, delete node directly
 - Simply delete the node.
 - Set its parent's pointer (left/right) to nullptr.
 2. If the node has one child:
 - Link the parent of the node to the node's only child.
 - Delete the node.
 3. If the node has two children (left and right):
 - Find the inorder successor ,the smallest value in the right subtree .
 - Copy the successor's value into the node to delete.
 - Delete the successor node



Ejemplo de graphs:

- Tienen dirección pueden ir o regresar o puede ser solo ida, depende de las representaciones
- Undirected: de un vertex se puede ir y regresar de otro
- Directed: a veces cuando vas a un vertex ya no te puedes regresar.

Implementation Hash tables:

```
#include <iostream>
#include <list>
using namespace std;
```

```
class Hashing {
private:
int buckets;
list<int>* table;
public:
Hashing(int buckets) {
```

```
this->buckets = buckets;
table = new list<int>[this->buckets];
}
void insert(int value) {
int index = value % buckets;
table[index].push_back(value);
}
void print() {
for (int i = 0; i < buckets; i++) {
cout << "Buckets: " << i << "|";
for (auto j : table[i])
cout << j << "->";
cout << "NULL" << endl;
}
}
};
```

```
int main() {
Hashing ht(10);
ht.insert(100);
ht.insert(95);
ht.insert(50);
ht.insert(112);
ht.insert(75);
ht.insert(25);
ht.insert(150);
ht.print();
}
```